Collaborative Defense in Industrial Data Spaces: A Quantum-Resistant Federated Learning Approach for Zero-Trust Edge Gateways

Author Name(s) — placeholder
Affiliation — placeholder
City, Country
email@example.org

Abstract*—The proliferation of the Industrial Internet of Things has dissolved traditional network perimeters, and Zero-Trust Architectures (ZTA) respond by demanding continuous verification of every request. At the industrial edge, however, Web Application Firewalls still rely on static signature sets that cannot generalise to unseen payloads, while the obvious remedy—shipping raw HTTP logs to a central model—violates the data sovereignty requirements of Industrial Data Spaces. We present a Federated Learning (FL) architecture for adaptive Zero-Trust edge gateways in which each gateway trains a lightweight linear classifier over hashed character n-grams of its own traffic and exchanges only model parameters with a cloud aggregator. Train and test sets are disjoint at the level of unique attack payloads. We find that the benefit of federation depends on how heterogeneity is distributed: when nodes differ in their benign traffic profile while facing a shared threat landscape, federation matches the strongest isolated silo within run-to-run variance without ever sharing raw data, and its main value is cross-domain generalisation for the weakest silo. Coordinate-wise median aggregation preserves detection quality when two of seven nodes upload sign-flipped weights, a setting in which plain federated averaging collapses. The control plane is hardened with signed, nonce-protected, rate-limited updates from enrolled gateways, and the weight channel is designed around ML-KEM post-quantum key encapsulation, which we benchmark as a component but have not yet wired into the live update path. On a Raspberry Pi 4, the pure-Python inferencer runs in a few hundred microseconds per request within a roughly 10 MB footprint, and each synchronisation transfers a fixed 1,040 bytes irrespective of inspected traffic volume, cutting cloud-bound bandwidth by up to 99.9% at the measured throughput with a five-minute synchronisation interval (98.9% at the deployed 15 s interval). Detailed measurements are reported in Section IV.

***Index Terms*—Federated Learning, Zero-Trust, Edge Computing, Industrial IoT, Byzantine Robustness, Post-Quantum Cryptography, Data Sovereignty**

I. Introduction

The move towards Industry 4.0 has interconnected large numbers of cyber-physical systems, and with them has widened the attack surface well beyond what a hardened network perimeter can contain. Perimeter security assumes that actors inside the local network are trustworthy. Zero-Trust Architecture (ZTA) discards that assumption and requires every request to be authenticated and authorised on its own merits, continuously and regardless of origin [1], [2]. At the industrial edge, the enforcement point is typically a gateway that terminates HTTP traffic for web front-ends and device APIs, so the quality of the inspection performed there largely decides how much zero trust is realised in practice.

Making that inspection adaptive is hard. Web Application Firewalls (WAFs) and intrusion detection systems deployed on gateways are mostly signature-based [3]–[5]. They need manual rule updates and generalise poorly to polymorphic or previously unseen payloads. Machine learning offers an alternative, but a robust model conventionally requires aggregating raw traffic in one place. In an Industrial Data Space such as Gaia-X [6], forwarding HTTP requests that embed proprietary API queries, authentication tokens and process telemetry is exactly what the governance model forbids. Federated Learning (FL) [7] removes this obstacle by moving computation to the data: each gateway trains on its own traffic and shares only model parameters. A gateway at one site can then benefit from an attack first seen at another without any raw payload leaving either.

Three concerns that are usually treated separately meet in this setting.

a) Zero-trust authentication of the learning loop itself.: If the gateways enforce zero trust on user requests, the control plane that distributes their models must meet the same standard. A federated aggregator that accepts unauthenticated updates lets an attacker register arbitrarily many sybil identities, and no Byzantine-robust aggregation rule [8], [9] tolerates an unbounded number of malicious clients. Authentication of participants is therefore a precondition for robust aggregation, not an add-on.

b) Federation across heterogeneous sites.: FL is often presented as a free improvement, but convergence under non-IID data can be worse than local training when client objectives diverge [10]. Whether federation pays off at a given site depends on how its data differs from its peers'. This is rarely measured for WAF traffic.

c) Post-quantum protection of the model channel.: Model updates are sensitive: gradients can leak training inputs [11], and a model channel that is recorded today can be decrypted later by a quantum-capable adversary ("store now, decrypt later"). Lattice-based key encapsulation, standardised as ML-KEM in FIPS 203 [12], addresses this, but its cost on

constrained edge hardware needs to be established rather than assumed.

A. Problem statement

We consider K edge gateways, each observing HTTP request traffic that must remain on site, and one cloud aggregator. The task is to obtain a request classifier that (i) separates benign requests from injection attacks (SQL injection and cross-site scripting) better than what any single gateway can learn alone, or at least without losing to it; (ii) runs inside the latency, memory and dependency budget of an ARM gateway; (iii) exchanges only fixed-size model parameters with the cloud; and (iv) stays correct when some participants are malicious, so that the aggregator must authenticate every update and bound the damage a compromised, enrolled gateway can do. We ask when federation actually helps, whether the learning loop can be made zero-trust end to end, and what it costs on real edge hardware.

B. Contributions

This paper makes the following contributions.

1) **Federated zero-trust gateway.** We design and implement a gateway architecture in which each edge node scores requests with a linear classifier over hashed character n-gram features. The feature transform is a single pure-standard-library module shared by cloud and edge, and the inferencer needs no third-party packages (Section III).
2) **When federation helps.** We show that the axis along which the data is non-IID, and not merely its presence, decides whether federation beats isolated training. Splitting nodes by threat campaign yields a federated model that lands between the silos, whereas splitting by benign traffic profile under a shared threat landscape yields one that matches or exceeds them, with train and test sets disjoint at the level of unique payloads (Section IV).
3) **Authenticated, Byzantine-robust aggregation.** We replace federated averaging with coordinate-wise median aggregation behind an authenticated control plane: per-gateway HMAC-SHA256 signatures with timestamps and single-use nonces, an enrolment allowlist, per-identity rate limiting, and dimension, finiteness and norm checks. With 7 nodes of which 2 upload sign-flipped weights, the median keeps ROC-AUC at 0.984 while plain averaging collapses to 0.263. A ten-case attack suite exercises the control plane against a live cluster, and the aggregator reports the Byzantine tolerance it actually achieves rather than implying protection it cannot give (Section V).
4) **Post-quantum transport.** We benchmark ML-KEM-512 on a Raspberry Pi 4, where encapsulation takes 0.162 ms, and argue that post-quantum protection of the weight channel is effectively free next to the synchronisation interval. We state plainly that ML-KEM is evaluated as a component and is not yet wired into the live channel (Sections V and IV).
5) **Edge cost and bandwidth.** On Cortex-A72 hardware the gateway sustains 452 requests per second at 0.229 ms per classification within a 10.4 MB resident set. Each synchronisation transfers a fixed 1,040-byte update regardless of inspected volume, which cuts upstream bandwidth by 99.94% at the measured throughput with a 300 s sync interval relative to shipping the requests themselves (Section IV).
6) **Methodology and honest scope.** We document two dataset-construction pitfalls that inflate results in this domain, namely resampling before splitting on unique payloads and rendering the two classes with different request templates, and we state the limits of the prototype: differential privacy is specified but not implemented, and detection is measured on the HTTP request surface described in Section IV.

C. Paper outline

Section II reviews related work on signature-based and learned detection, federated learning under heterogeneity, Byzantine-robust aggregation and post-quantum cryptography. Section III presents the system model, the feature pipeline and the gateway and aggregator architecture. Section IV describes the testbed and datasets and reports detection, federation, Byzantine-resilience, hardware and bandwidth results together with limitations. Section V defines the threat model and covers poisoning defence, control-plane authentication and the post-quantum channel. We then conclude.

II. Background and Related Work

This section fixes the concepts the rest of the paper relies on: zero-trust access control, federated learning, behavioural n-gram profiling of request payloads, and post-quantum cryptography for the weight channel. It then places the work against prior art.

A. Zero-Trust Architecture

Zero trust drops the assumption that a request is benign because it originates inside the network perimeter. NIST SP 800-207 formalises this: no implicit trust is granted by network location, and every access decision is made per request, based on the observable state of the requester and the resource [1]. Google's BeyondCorp was an early large-scale deployment of the idea [13]. The architecture separates a policy decision point from policy enforcement points that sit in front of each resource.

In our setting the enforcement point is a gateway-resident Web Application Firewall (WAF) on each edge node, and the decision signal is a learned score over the request payload. The open problem is how to keep that decision function current against new payloads without centralising the traffic it is trained on. Signature engines such as Snort [4] depend on curated rules and cannot adapt on their own. Sommer and Paxson also caution that learned detectors in network security face a hard "closed world" problem, because the benign class is site-specific and attacks keep changing [14].

This is the property that makes the benign-traffic profile, rather than the attack mix, the axis that matters for federation in our evaluation.

B. Federated Learning

Federated learning (FL) trains a shared model while data stays on the participants. In Federated Averaging (FedAvg), each client runs local optimisation and the server averages the returned parameters, weighted by local sample count [7]. Kairouz *et al.* survey the open problems [15]; those most relevant here are statistical heterogeneity, communication cost, and adversarial participants.

a) Heterogeneity.: When client distributions differ, local optima diverge. Li *et al.* prove that FedAvg converges more slowly on non-IID data [10], and FedProx adds a proximal term to stabilise training under such heterogeneity [16]. We do not use a proximal term; we instead study which *kind* of heterogeneity helps or hurts a plain FedAvg aggregation.

b) Privacy.: Raw data never leaves a client, but updates are not free of information: gradients can be inverted to reconstruct training inputs [11]. Differentially private training with clipping and noise [17] and cryptographic secure aggregation [18] are the standard mitigations. Mothukuri *et al.* survey the resulting attack and defence landscape [19].

c) Byzantine robustness.: A compromised client can upload arbitrary parameters. Krum [8], coordinate-wise median and trimmed mean [9] replace the mean with a robust statistic, under the assumption that honest clients outnumber malicious ones. These rules are not a complete defence: Fang *et al.* construct local model-poisoning attacks that degrade them [20].

d) Communication.: Update size dominates cost on constrained links. Quantisation and encoding schemes such as QSGD reduce it [21]. A fixed-width feature map attacks the same problem differently, by making the model, and hence the update, a constant size.

e) FL for IoT security.: FL has been applied to anomaly detection on IoT gateways, for example DIoT, which learns per-device-type communication profiles federatedly [22], and is surveyed for IoT in general by Nguyen *et al.* [23]. We are not aware of prior work that evaluates, as we do, how the choice of non-IID axis (attack campaign versus benign profile) affects a federated WAF.

C. Behavioural and n-gram Profiling of Payloads

Behavioural profiling models what normal input looks like and scores deviations. Forrest *et al.* used short sequences of system calls as a signature of "self" for processes [24]. For network and web traffic, PAYL models the byte-frequency distribution of payloads [25], and Kruegel and Vigna model request attributes of web applications [26]. Character n-gram statistics are a long-standing representation in text classification [27], and they transfer directly to HTTP payloads because injection attacks have distinctive local character patterns (for example quote–keyword–comment sequences in SQL injection, or angle-bracket–tag sequences in XSS).

An explicit n-gram vocabulary grows with the traffic seen, which would make the model size, and every synchronisation payload, unbounded. The hashing trick maps n-grams into a fixed number of buckets without a vocabulary [28]. Our feature map hashes character trigrams ($n = 3$) into 256 buckets with a polynomial rolling hash, ℓ_2-normalises the bucket counts, and appends three scaled syntactic scalars (length, count of non-alphanumeric characters, Shannon entropy), giving a 259-dimensional vector (`fedzta_features.py`). The same module runs on cloud and edge so both sides bucket identically. Because the map is fixed and cheap to compute, a linear classifier suffices on the gateway, in contrast to the deep intrusion detectors whose footprint embedded gateways struggle to carry.

D. Post-Quantum Cryptography for the Weight Channel

Model parameters are valuable, and an adversary who records the channel today may decrypt it later. Shor's algorithm breaks the factoring and discrete-logarithm problems that underlie RSA and elliptic-curve Diffie–Hellman [29], so key exchange that relies on them is exposed to this "store now, decrypt later" strategy. Post-quantum cryptography replaces them with problems believed hard for quantum computers [30].

NIST has standardised two lattice-based schemes derived from the CRYSTALS submissions. **ML-KEM** (FIPS 203) [12], derived from CRYSTALS-Kyber [31], is a key-encapsulation mechanism: the receiver publishes a public key, the sender encapsulates a fresh shared secret under it and returns a ciphertext, and the receiver decapsulates. The shared secret then keys a symmetric cipher for the payload. **ML-DSA** (FIPS 204) [32], derived from CRYSTALS-Dilithium [33], is the corresponding digital signature scheme.

Our prototype uses ML-KEM-512, the smallest parameter set, through the `pqcrypto` Python bindings. The cloud aggregator (`cloud_pqc.py`) generates a keypair and serves the public key. The edge node (`edge_pqc.py`) fetches it and encapsulates a shared secret, which protects the model-weight transfer. Message authentication is handled separately. Each gateway is enrolled with a 256-bit secret and signs updates with HMAC-SHA256 (`fedzta_auth.py`), which uses only the Python standard library and keeps the edge footprint small. A signature scheme such as ML-DSA would replace the shared secret with per-gateway public keys, at the price of an extra dependency on the edge. We discuss this as a design alternative and do not evaluate it.

E. Positioning

Compared with this literature, our contribution is the combination rather than any single component. We (i) cast the federated WAF as a zero-trust enforcement point with a fixed-size, hashed n-gram model that is cheap enough for ARM gateways; (ii) separate two non-IID axes and show empirically that only one of them makes federation outperform local training; and (iii) protect the weight channel with a NIST-standardised post-quantum KEM and authenticate updates with a lightweight

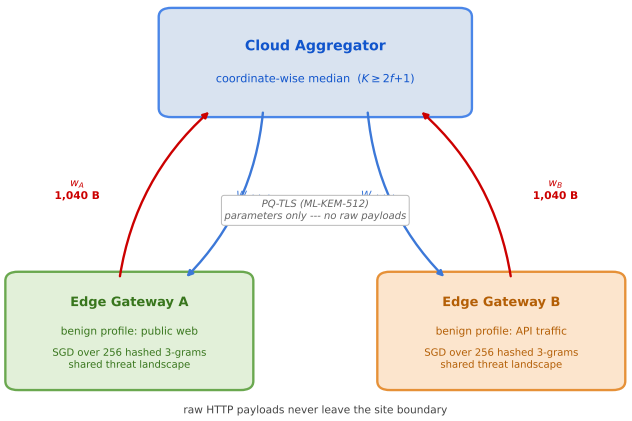


Fig. 1. FedZTA-Edge architecture. Edge gateways score requests locally and upload signed weight vectors; the cloud aggregator authenticates, gates, and robustly aggregates them. Raw traffic never leaves the edge.

MAC. The robustness we claim is limited to the baseline Byzantine rules above, and no formal privacy guarantee is claimed; both limits are discussed with the evaluation.

III. System Architecture and Method

FedZTA-Edge splits a Zero-Trust web application firewall (WAF) into two roles. Each *edge gateway* inspects every request on an ARM IoT node with a small linear classifier and never transmits payloads. A *cloud aggregator* receives only authenticated model parameters, combines them with a Byzantine-robust rule, and returns a global model. Figure 1 shows the data and control flow; the following subsections describe each component and name the source module that implements it.

A. Threat Model and Design Constraints

We assume gateways with a Cortex-A72-class CPU, no GPU, and a Python standard-library runtime. Traffic is private: payloads cannot be shipped to the cloud. The network path between gateway and aggregator is adversarial; an attacker may observe, replay, reorder or forge update messages but holds no enrolled secret. A gateway that does hold a valid secret may be compromised and then behaves as a Byzantine client, which the aggregation rule, not the authentication layer, must contain (Section III-C). These assumptions drive the choice of a linear model, a dependency-free feature transform, and symmetric authentication.

B. Feature Extraction

All feature computation lives in one module, `fedzta_features.py`, imported verbatim by the gateway, the aggregator and the profiler, so that edge and cloud bucket n-grams identically. A request context P (a path, query string, header value or body) is mapped to a vector $\mathbf{x}(P) \in \mathbb{R}^{D+3}$ with $D = 256$, giving 259 dimensions:

$$\mathbf{x}(P) = \left[\, \hat{\mathbf{h}}(P) \;\middle|\; \frac{|P|}{100} \;\middle|\; \frac{s(P)}{20} \;\middle|\; \frac{H(P)}{5} \,\right]. \tag{1}$$

Here $\hat{\mathbf{h}}(P)$ is the ℓ_2-normalised histogram of lower-cased character 3-grams hashed into D buckets by a polynomial rolling hash with multiplier 131 over 32-bit state, $s(P)$ counts non-alphanumeric characters, and $H(P)$ is the Shannon entropy of the character distribution. A polynomial hash replaces MD5 because it is faster on the Cortex-A72 (0.242 ms versus 0.404 ms per request in an earlier profiling run; the later end-to-end measurement in Section IV-E is 0.229 ms) without measurable accuracy loss. The three scaling constants $(100, 20, 5)$ are fixed a priori and never fitted on data, so no scaler has to be shipped to the edge and no test statistics leak into training.

The classifier is logistic regression, $p(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, with the logit clipped to $[-250, 250]$ before the sigmoid. The full model is 259 coefficients plus an intercept, i.e. 1,040 bytes in single precision, which is also the size of one synchronisation payload.

C. Federated Training and Aggregation

a) Offline federated training.: The shipped model is produced by iterative federated averaging [7] over two simulated gateways (`fl_simulation_v4.py`, `train_production_model.py`). Each node holds an `SGDClassifier` with logistic loss, ℓ_2 penalty $\alpha = 10^{-4}$ and constant learning rate $\eta_0 = 0.01$. In every round each node starts from the current global parameters, trains for 30 local epochs on its own data, and the server forms the sample-weighted average

$$\theta^{(t+1)} = \sum_k \tfrac{n_k}{n}\, \theta_k^{(t)}, \qquad n = \sum_k n_k, \tag{2}$$

for 20 rounds. Train and test sets are disjoint at the level of unique payloads, and all benign and malicious requests share one request template. The training configuration, including the topology in which nodes differ by benign traffic profile (public web versus API gateway), is the one selected in Section IV.

b) Online robust aggregation.: In deployment the aggregator (`robust_cloud.py`) keeps only the latest update per enrolled client and recomputes the global model on every accepted update. With K stored updates the default rule is the coordinate-wise median [9],

$$w_j = \operatorname{median}\{w_{k,j}\}_{k=1}^{K}, \qquad b = \operatorname{median}\{b_k\}_{k=1}^{K}, \tag{3}$$

with a trimmed mean and a plain mean selectable by configuration for comparison. The median tolerates f Byzantine clients only while $K \geq 2f + 1$; the `/weights` endpoint therefore reports $f_{\max} = \max(0, \lfloor (K-1)/2 \rfloor)$ and a `robust` flag, so a deployment with $K < 3$ is visibly not protected.

Before an update enters the pool it must pass the checks in order: authentication (Section III-D), a per-client rate limit (default one update per 5 s), dimension and finiteness checks, and a *norm gate*. The gate rejects an update whose ℓ_2 norm, relative to a reference norm, falls outside $[1/\rho, \rho]$ with $\rho = 4$. The reference is the median norm of the other peers when at least two have reported, the larger of the single other peer's norm and the bootstrap model's norm when exactly one has, and the bootstrap model's norm otherwise, so the gate

stays active even at $K = 2$. Rejecting gross outliers before aggregation prevents a Byzantine client from consuming the per-coordinate honest majority on which the median depends.

D. Authenticated Updates

Byzantine robustness is vacuous if anyone can register arbitrarily many identities. `fedzta_auth.py` therefore binds every update to an enrolled gateway. Enrolment (`enroll_peer.py`) issues a 256-bit secret from the OS CSPRNG. An update is a body $\{\text{client_id}, \mathbf{w}, b, n, \text{ts}, \text{nonce}\}$ serialised canonically (sorted keys, fixed separators) and tagged with HMAC-SHA256 [34]. Verification checks, in this order, that the client is enrolled (unknown and malformed identifiers receive the same response), that the tag matches under a constant-time comparison, that the timestamp lies within ± 120 s, and that the nonce has not been seen within a 300 s window (`ReplayGuard`). Symmetric HMAC is chosen over a public-key signature because it needs only `hmac` and `hashlib` from the standard library, preserving the zero-dependency edge footprint.

E. Post-Quantum Channel

Weights are intellectual property, and a recorded channel can be decrypted later by a quantum adversary ("store now, decrypt later"). The channel is therefore designed around ML-KEM-512 [12]. In the prototype (`cloud_pqc.py`, `edge_pqc.py`) the aggregator generates a key pair and serves its public key at `/pubkey`; the gateway encapsulates against it, sends the ciphertext to `/exchange`, and both sides derive the same shared secret via decapsulation. The shared secret is intended to key authenticated encryption of the update body. In the current prototype the key exchange is demonstrated and benchmarked as a standalone component, and the live update path (`/update`, `/weights`) still carries HMAC-authenticated JSON over plain HTTP; integrating the derived key into that path remains future work, and authenticity today rests on HMAC alone.

F. Continuous Inspection Gateway

The gateway (`continuous_waf.py`) is a multi-threaded HTTP server. For each GET, POST or PUT request it builds a list of *contexts*: the URL path, the percent-decoded query string, the `User-Agent`, `Referer` and `Cookie` header values, and, for POST and PUT, the body truncated to 8 KB. Each context is scored independently with the current model and the request score is the maximum,

$$s(r) = \max_{P \in \mathcal{C}(r)} p\big(\mathbf{x}(P)\big), \tag{4}$$

so a payload hidden in any single field is not diluted by benign fields. The request is blocked (HTTP 403) when $s(r) \geq \tau$. Two operating switches support safe rollout: *shadow mode* scores and counts would-be blocks but always forwards, and the *failure policy* chooses between fail-open (default, return 200 on an internal error) and fail-closed. Sockets carry a 5 s timeout, and non-numeric `Content-Length` values are rejected and bodies are read only up to the 8 KB cap, bounding slow-client and oversized-body resource use. Counters are updated under a lock; `/health` and `/metrics` expose liveness and block counts.

a) Decision threshold.: The default τ is 0.5 in the offline profiler, but the deployed gateway uses $\tau = 0.15$. The shipped model was trained with an augmented, more varied benign distribution (JSON and form bodies, varied lengths, prose), which shifts the intercept downward ($b \approx -5.57$) and lowers attack-class probabilities; τ was lowered to 0.15 to compensate. Section IV reports operating points that do not depend on this single threshold.

b) Anchor set instead of pseudo-labelling.: An earlier design trained online on the gateway's own confident predictions. Self-labelling lets the model reinforce its own errors and drift, so it was removed. Local adaptation now happens only by replaying a verified *anchor set* (`anchor_set.json`), a small list of labelled payloads checked by an operator, through one SGD step each ($\eta = 0.05$) every 60 s. The anchor set is a drift guard and a hook for operator-verified labels rather than a source of new knowledge; new knowledge arrives through federation.

c) Synchronisation.: Every 15 s the gateway uploads a signed snapshot (weights, bias, sample count), downloads the global model, and applies it if its length equals 259, persisting it atomically (write, `fsync`, rename). Rejections (401 unauthenticated, 409 norm gate, 429 rate limit) are logged and the gateway keeps its current model, so a cloud outage never stops inspection.

G. End-to-End Procedures

Algorithm 1 summarises per-request inspection and synchronisation on the edge; Algorithm 2 summarises update handling on the cloud.

H. Resource Footprint

Every message in the update protocol has fixed size set by the model dimension: one update carries 260 values (1,040 bytes in single precision), independent of the number of inspected requests. Measured latency, memory, throughput and bandwidth are reported in Section IV.

IV. Evaluation

We evaluate FedZTA-Edge along five questions. (i) Does federation improve detection over isolated edge models, and under which kind of heterogeneity? (ii) How accurate is the shipped, augmented model on payloads it has never seen? (iii) Does robust aggregation survive model poisoning? (iv) What does the gateway cost on ARM hardware? (v) How much cloud-bound traffic does federation save? Every number below is read from a result file or script in the repository (`matrix_results.json`, `fed_model.json`, `poisoning_results.json`, `bandwidth_results.json`); where a number is not backed by a current result file, we say so. We report weaknesses alongside strengths, and Section IV-G collects them.

Algorithm 1 Edge gateway: inspection and synchronisation

Require: model $(\mathbf{w}, b)$, threshold τ, anchor set $\mathcal{A}$, secret k, interval T

procedure INSPECT(request r)
  $\mathcal{C} \leftarrow$ path, decoded query, `User-Agent`, `Referer`, `Cookie`, body[:8192]
  $s \leftarrow \max_{P \in \mathcal{C}} \sigma(\mathbf{w}^\top \mathbf{x}(P) + b)$
  if $s \geq \tau$ **and not** shadow mode **then return** 403
  else return forward
  end if
end procedure
procedure SYNCLOOP
  while true **do** wait T seconds
    $m \leftarrow (\text{BODY}(\text{id}, \mathbf{w}, b, n, \text{ts}, \text{nonce}),\ \text{HMAC}_k(\cdot))$; POST m to cloud
    $(\mathbf{w}_g, b_g) \leftarrow$ GET global model
    if $|\mathbf{w}_g| = 259$ **then** $(\mathbf{w}, b) \leftarrow (\mathbf{w}_g, b_g)$; persist atomically
    end if
  end while
end procedure
procedure ANCHORLOOP
  while true **do** wait 60 s
    for all $(P, y) \in \mathcal{A}$ **do** $\mathbf{w} \leftarrow \mathbf{w} - \eta(\sigma(\mathbf{w}^\top \mathbf{x}(P) + b) - y)\,\mathbf{x}(P)$ (and b likewise)
    end for
  end while
end procedure

TABLE I
UNIQUE PAYLOADS AFTER DEDUPLICATION AND THE DISJOINT 70/30 TRAIN/TEST SPLIT (`FED_MODEL.JSON`).

Class	Unique	Train	Test
Benign (web)	43,007	30,104	12,903
Benign (API)	8,013	5,609	2,404
SQLi	681	476	205
XSS	523	366	157

A. Experimental Setup

a) Testbed.: The cloud aggregator is an x86_64 Ubuntu server (14 cores, 31 GB RAM) running the aggregation service and all offline training studies. Two edge gateways are Raspberry Pi 4 boards (aarch64 Cortex-A72, 4 GB RAM), each running the pure-Python threaded gateway (`continuous_waf.py`) on port 8080 with a 15 s synchronisation interval. Gateways identify themselves with a stable client identifier so the aggregator can key updates by peer.

b) Data.: All payloads come from the public SecLists corpus [35]; no payload is synthesised. Attacks are SQL injection (SQLi, 17 lists) and cross-site scripting (XSS, 7 lists). Benign traffic has two profiles: "web" (path words, `raft-small-words`) and "API" (`api-seen-in-wild`, `api-endpoints`, `graphql`). Table I lists the deduplicated corpora.

Algorithm 2 Cloud aggregator: handling an update

Require: enrolled secrets $\{k_c\}$, replay guard, norm limit ρ, rate interval δ

procedure ONUPDATE(message m)
  if not VERIFY$(m, \{k_c\})$ **then return** 401
  end if ▷ enrolled, HMAC, timestamp, nonce
  if client updated less than δ ago **then return** 429
  end if
  if $|\mathbf{w}| \neq 259$ **or** non-finite **then return** 400
  end if
  ref $\leftarrow$ median norm of other peers (or bootstrap norm fallback)
  if $\|\mathbf{w}\|_2/\text{ref} \notin [1/\rho, \rho]$ **then return** 409
  end if
  latest$[c] \leftarrow (\mathbf{w}, b)$
  $w_j \leftarrow \text{median}_k\ w_{k,j} \quad \forall j; \quad b \leftarrow \text{median}_k\, b_k$; publish version
  return 200
end procedure

Two protocol decisions matter more than any hyperparameter. First, we split by *unique payload* before any resampling: drawing train and test sets independently from one pool places identical strings in both and measures memorisation. With a few hundred unique attack strings the effect is severe, so every test payload is absent from every node's training data. Second, benign and malicious values are placed in the same request context, so that request structure alone cannot predict the label; an earlier configuration in which benign and attack requests used different templates let a classifier reach a perfect ROC-AUC on attack classes it never trained on, by separating URL shapes rather than maliciousness.

c) Data augmentation.: Early gateway models learned payload *length* and surrounding context rather than content: attacks in the corpora are short, so benign requests with realistic headers, JSON bodies or long prose were flagged. The production training script (`train_production_model.py`) therefore wraps every sampled payload, benign or malicious, in one of seven contexts chosen uniformly: the bare value; a value prefixed with a browser user-agent string; a cookie header with random identifiers; a full URL with a query string; a JSON body with a prose padding field; a form body with a CSRF token; and a prose sentence preceding the value. This is intended to neutralise the length feature and the benign-header baseline. It also shifts the model's intercept, which is why the gateway's blocking threshold was lowered to `BLOCK_THRESHOLD` $= 0.15$ (`continuous_waf.py`).

d) Node topologies.: Each node trains on 3,000 benign and 1,000 attack samples (500 SQLi, 500 XSS); the test set has 1,500 benign (750 web, 750 API) and 1,000 attack samples (500 per class), all drawn from held-out payloads. We study two non-IID axes. In the *attack* axis both nodes share the benign profile but see disjoint attack mixes (the matrix study uses disjoint campaigns). In the *domain* axis both nodes see the full attack mix but different benign profiles: node A web

paths, node B API endpoints.

e) Features and training.: A request is mapped to a vector in two ways. *Syntactic* features are three scalars (length, count of non-alphanumeric characters, Shannon entropy), each scaled by a fixed a-priori constant. *n-gram* features are ℓ_2-normalised hashed character 3-gram counts in D buckets (polynomial rolling hash) followed by the three syntactic scalars, giving dimension $D + 3$. The classifier is logistic regression trained by SGD (`loss=log_loss`, ℓ_2 penalty $\alpha = 10^{-4}$, constant $\eta_0 = 0.01$). Federated training runs $R = 20$ rounds of $E = 30$ local epochs, with sample-weighted averaging; isolated baselines train for $R{\cdot}E$ epochs on their own data. Results are averaged over eight seeds (42–49), each with a fresh payload split; we report mean $\pm$ standard deviation.

f) Metrics.: We report ROC-AUC and PR-AUC; recall on each attack class at a matched 0.1% false-positive budget on the pooled benign test set (so models at different operating points are comparable); and the false-positive rate on each benign domain at the default decision threshold (probability 0.5). The "FPR" columns therefore describe the default threshold, *not* the gateway's deployed threshold of 0.15.

g) Two experimental configurations.: The design-space study (`fl_simulation_v4.py`, output `matrix_results.json`) compares both feature sets across both axes using a small $D = 64$ (dim $= 67$) hashing and unaugmented, templated requests; it is a controlled comparison, not the shipped model. The production study (`train_production_model.py`, output `fed_model.json`) trains the shipped $D = 256$ (dim $= 259$) model with the augmentation above. They use different data pipelines and should not be compared numerically.

B. Where Does Federation Help?

Table II gives the full feature $\times$ axis matrix.

The pattern is consistent, and less flattering than a federation advocate would like. On the attack axis, each silo is blind to the other's attack class, and federation recovers most of the gap for the weaker node. With n-gram features Edge A's SQLi recall rises from 0.581 (isolated) to 0.764 (federated). The federated model remains below the stronger silo (Edge B, 0.805), which is the behaviour expected of FedAvg when local optima diverge [10]: the average lies between the two, not above. On the domain axis the federated model is statistically indistinguishable from the better silo in ROC-AUC (0.9918 vs. 0.9916 with n-grams; the standard deviation across seeds is 0.003) and is below it on SQLi recall (0.833 vs. 0.850). Its benefit is that it is as good as the *best* silo on *both* benign domains without knowing in advance which silo that is: Edge A alone has a 1.0% default-threshold FPR on API traffic, which federation reduces to 0.10% with n-grams. In short, federation reliably lifts the weaker node and lowers cross-domain false positives. It does not, in these experiments, reliably beat the best isolated model.

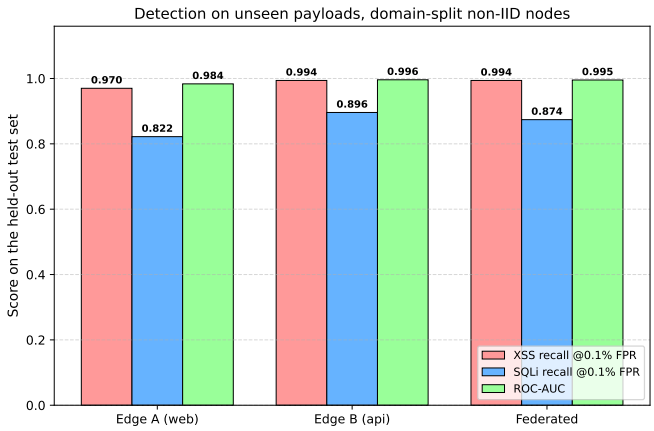


Fig. 2. Per-class detection of the shipped model for a single seed (seed 49, the last one run; `fed_model.json` stores this run's coefficients and metrics although its config records 42 as the shipped seed); the multi-seed means are in Table III. A single seed is not representative of the spread: this seed's federated model happens to beat both silos on all three metrics.

C. Shipped Model Accuracy

Table III reports the production configuration ($D = 256$, augmented, domain split).

The federated model has the highest mean ROC-AUC (0.9947) and the lowest ROC-AUC variance across seeds, but its margin over Edge B (+0.0009) is a fraction of one standard deviation. A per-seed comparison supports the same reading: federated ROC-AUC exceeds Edge B in 5 of 8 seeds and Edge A in 8 of 8. On SQLi recall, the hardest class, Edge B is higher on average (0.879 vs. 0.865), and the federated model is worse by more than 0.08 in the worst seed. Federated XSS recall (0.980) matches Edge B (0.979). We therefore do not claim that federation beats the best silo; we claim it matches it, beats the silo that has the wrong benign distribution, and has the lowest false-positive rate on web traffic. Fig. 2 (one seed) is more favourable than the multi-seed mean and should be read with that in mind. The disagreement between earlier single-seed runs about whether federation "wins" is precisely what the seed-stability study (`seed_stability.py`) was written to expose; the multi-seed results above supersede any single-run claim.

a) Convergence.: Global ROC-AUC (last seed, 49) is 0.9872 after round 1, 0.9943 after round 3 and 0.9958 after round 20 (`fed_model.json`), so most of the benefit arrives within a few rounds. Rapid convergence bounds how long a newly learned pattern takes to propagate between sites.

b) Accuracy versus update size.: Fig. 3 (right) sweeps the hashing dimension $D \in \{32, \ldots, 512\}$ (`sweep_ngram.py`); an update costs $4(D + 4)$ bytes, e.g. 1,040 B at $D = 256$ and 2,064 B at $D = 512$. *Caveat:* the sweep values plotted in `fl_sweep.pdf` are hardcoded in `make_figures.py` from an earlier, pre-augmentation run; they have not been regenerated with the current pipeline (the federated ROC-AUC varies by under 0.007 across all D, i.e. the curve is nearly flat). We include the figure for the qualitative message that

TABLE II

DESIGN-SPACE MATRIX (`MATRIX_RESULTS.JSON`; MEAN OVER EIGHT SEEDS; STD. OF ROC-AUC IN PARENTHESES). RECALL COLUMNS ARE AT 0.1% POOLED FPR. FPR-API IS THE DEFAULT-THRESHOLD FALSE-POSITIVE RATE ON API TRAFFIC. BOLD MARKS THE BEST ROC-AUC IN EACH FEATURE/AXIS GROUP; DIFFERENCES UNDER 0.0005 ARE TREATED AS TIES AND BOTH ARE MARKED.

Features	Axis / model	ROC-AUC	XSS	SQLi	FPR-API
Syntactic (3)	attack: Edge A	0.9723 (0.0055)	0.894	0.679	0.0140
	attack: Edge B	**0.9891** (0.0034)	0.922	0.777	0.0123
	attack: Federated	0.9788 (0.0050)	0.903	0.737	0.0080
	domain: Edge A	0.9803 (0.0054)	0.902	0.736	0.0085
	domain: Edge B	**0.9919** (0.0033)	0.946	0.841	0.0010
	domain: Federated	0.9902 (0.0034)	0.926	0.809	0.0012
n-gram, D=64 (67)	attack: Edge A	0.9532 (0.0056)	0.904	0.581	0.0135
	attack: Edge B	**0.9895** (0.0031)	0.913	0.805	0.0098
	attack: Federated	0.9826 (0.0044)	0.927	0.764	0.0053
	domain: Edge A	0.9839 (0.0043)	0.915	0.768	0.0102
	domain: Edge B	**0.9916** (0.0035)	0.954	0.850	0.0013
	domain: Federated	**0.9918** (0.0032)	0.952	0.833	0.0010

TABLE III

SHIPPED $D = 256$ AUGMENTED MODEL ON UNSEEN PAYLOADS, DOMAIN-SPLIT NODES (`FED_MODEL.JSON`), MEAN ± STD OVER EIGHT SEEDS. RECALL AT 0.1% POOLED FPR; FPR AT THE DEFAULT THRESHOLD.

Model	ROC-AUC	PR-AUC	XSS	SQLi	FPR web	FPR API
Edge A	0.9859±0.0035	0.9871±0.0032	0.954±0.046	0.807±0.073	0.0010	0.0052
Edge B	0.9938±0.0036	0.9940±0.0031	0.979±0.020	**0.879**±0.033	0.0008	0.0005
Federated	**0.9947**±0.0019	**0.9950**±0.0017	**0.980**±0.022	0.865±0.059	0.0005	0.0007

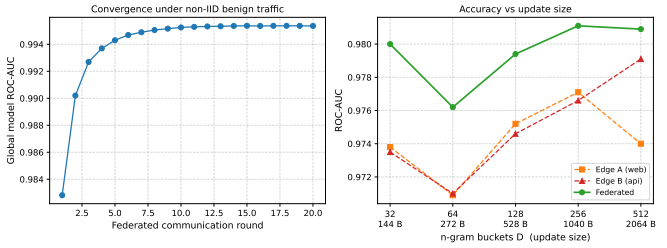


Fig. 3. Left: convergence of the global model's ROC-AUC across rounds. Right: ROC-AUC versus hashing dimension D (and update size). The right panel uses values from a pre-augmentation run and is retained for its qualitative message only.

accuracy is insensitive to D over a 16× range, and flag it for regeneration before submission. The left panel (convergence) is regenerated from the current `fed_model.json`.

D. Byzantine Resilience

We instantiate $K = 7$ nodes on the domain-split n-gram configuration (dim $= 259$, seed 42, 12 rounds, 20 local epochs). From round 4, $f = 2$ nodes upload $-12\,w_k$ in place of their weights ($f < K/2$ with margin). Each round the same uploads are aggregated by mean, coordinate-wise median and trimmed mean ($\beta = 2$) and each global model is scored on the held-out set (`poisoning_results.json`).

Plain averaging falls from 0.9841 ROC-AUC (round 3) to 0.0137 in the first poisoned round, i.e. the global model is inverted, then oscillates between 0.03 and 0.73 and ends at 0.263 (Table IV). Both robust rules keep improving after the attack begins, because the two poisoned uploads fall in the

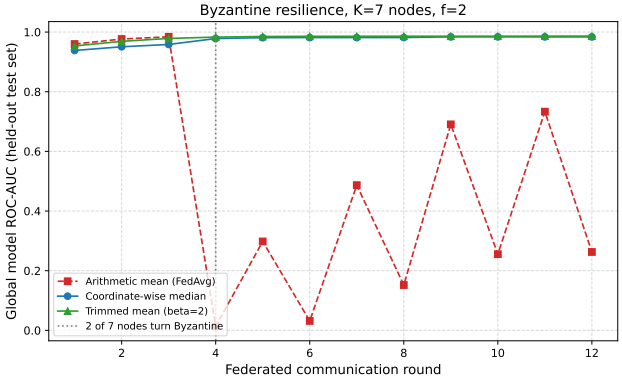


Fig. 4. Measured poisoning resilience (`poisoning_experiment.py`). From round 4, two of seven nodes upload sign-flipped, 12× amplified weights.

TABLE IV

GLOBAL-MODEL ROC-AUC UNDER POISONING (`POISONING_RESULTS.JSON`); THE ATTACK STARTS IN ROUND 4.

Aggregator	Round 3	Round 4	Round 12
Mean (FedAvg)	0.984	0.014	0.263
Coordinate median	0.958	0.978	0.984
Trimmed mean (β=2)	0.979	0.983	0.986

trimmed tails of every coordinate. Median ends at 0.9838 and trimmed mean at 0.9864. This is a single seed and a naive sign-flip adversary. Optimised attacks that know the aggregation rule can defeat median-style defences [20], and we claim no resistance to them. The experiment also assumes updates are authenticated (the gateways sign updates), since an unauthenticated aggregator has unbounded f.

E. Edge Cost on ARM

Provenance. The measurements in this subsection were taken on the Raspberry Pi testbed and are recorded in the earlier draft (`federated_paper_v5.tex`), not in a current JSON result file; the load generator is `stress_test.py`. They were taken with the 259-dimensional model but before

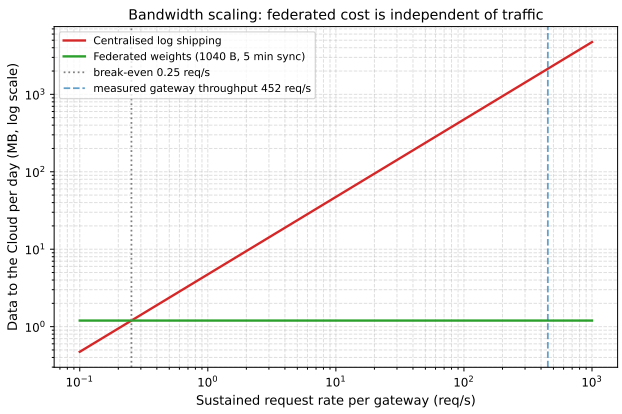


Fig. 5. Daily traffic to the cloud versus sustained request rate (`bandwidth_analysis.py`). The federated cost is flat because each synchronisation carries a fixed 1,040 B update regardless of inspected volume.

the final augmentation and the threshold change, which do not alter inference cost (feature dimension and arithmetic are unchanged). They should be re-measured and archived before submission.

Inference with the 259-coefficient model averages 0.229 ms per request (up from 0.042 ms with three features), a single-core ceiling of about 4,370 req/s, with a peak resident set of 10.4 MB dominated by the Python interpreter rather than the 1,040 B model. Under 1,000 requests at concurrency 10, repeated ten times, the threaded gateway sustained 452.5 ± 24.3 req/s, median latency 20.5 ± 0.7 ms, 95th percentile 32.2 ± 2.5 ms, and no failed requests over 10,000. Throughput is an order of magnitude below the inference ceiling, so the HTTP stack, not the classifier, is the bottleneck. A single-threaded server serialised keep-alive connections and capped throughput at 42–56 req/s with 2.7–3.6% timeouts; moving to `ThreadingHTTPServer` removed this, an artefact easily mistaken for a machine-learning cost. These throughput figures are for *benign* traffic to a gateway issuing HTTP 200s; we have not measured throughput under a mixed attack load.

ML-KEM-512 on the same board costs 0.122 ms (key generation), 0.162 ms (encapsulation) and 0.205 ms (decapsulation) over $n = 1000$ warmed-up trials (std. ≤ 0.003 ms), negligible against a 15 s synchronisation interval. Single-shot timings taken immediately after process start overstate these by 60% or more, which an earlier revision reported.

F. Bandwidth and Data Sovereignty

Bandwidth claims for federated learning often compare a whole dataset against one update from one node. We charge federation for every round, every node and both directions. The 20-round training run transfers $20 \times 2 \times 2 \times 1{,}040 = 83{,}200$ B against 332,983 B of raw request payloads (mean request 27.4 B): a reduction of only 75.0%. The operational case is stronger because the costs scale differently: shipping logs grows linearly with traffic, while federation depends only on the synchronisation interval (Table V).

TABLE V
DAILY CLOUD-BOUND TRAFFIC FOR TWO GATEWAYS (`BANDWIDTH_RESULTS.JSON`).

Rate	Sync	Centralised	Federated	Reduction
10 req/s	300 s	47.3 MB	1.2 MB	97.47%
100 req/s	300 s	473.1 MB	1.2 MB	99.75%
452 req/s	300 s	2,138.4 MB	1.2 MB	99.94%
452 req/s	15 s	2,138.4 MB	24.0 MB	98.88%

At the 15 s interval the deployed testbed actually uses, the reduction at 452 req/s is 98.88%. The break-even is about 0.25 req/s per node at a 5-minute interval, below which log shipping is cheaper. The comparison assumes the centralised baseline ships only the request payload (27.4 B on average), a conservative baseline, since real logs carry headers and metadata and would be larger. Beyond volume, federation keeps operational payloads inside the site boundary, the property Industrial Data Space governance requires.

G. Limitations of the Evaluation

We list the weaknesses a reader should weigh.

- **Small attack corpora.** Only 681 unique SQLi and 523 XSS strings exist. Per-seed SQLi recall at 0.1% FPR varies by ± 0.06, and the data are public fuzzing lists, not captured production attacks; absolute numbers will not transfer to adaptive attackers or to novel attack classes.
- **Federation gain is within noise.** Against the best silo, the ROC-AUC difference is below one standard deviation and SQLi recall is lower. The measured benefits are cross-domain false-positive reduction and protection of the weaker silo, not a higher ceiling.
- **Operating point.** FPR columns use threshold 0.5 while the gateway blocks at 0.15, a value chosen after augmentation shifted the intercept. We have no result file reporting detection or false-positive rate *at* 0.15 on held-out traffic; deployed false-positive behaviour is therefore not quantified here.
- **Augmentation is synthetic.** The seven wrappers are hand-designed. They remove the length shortcut we found but are not real traffic, and the model has not been evaluated on captured production logs.
- **Two nodes.** Detection experiments use $K = 2$ nodes; only the poisoning study uses $K = 7$.
- **Stale artefacts.** The D-sweep panel and the ARM latency/throughput/PQC figures are not backed by current result files (Sections IV-C and IV-E).
- **Poisoning.** One seed, one naive adversary; no optimised or backdoor attack.
- **Linear model.** A logistic regression on hashed 3-grams cannot capture long-range or semantic structure; evasions by encoding, fragmentation or obfuscation are untested.

V. Security Analysis

This section states what the system defends against, measures the poisoning defence, describes the authenticated control plane and the adversarial tests that exercise it, records the fixes made during a code audit, and closes with the limitations that bound every claim above.

A. Threat Model

We consider four adversaries.

- A1, application-layer attacker. Sends injection and cross-site-scripting payloads through an edge gateway toward upstream services. The gateway's classifier is the control under test.
- A2, network attacker. Observes, replays, reorders or forges messages on the edge–cloud path, and may retain ciphertext indefinitely ("store now, decrypt later"). A2 holds no enrolled secret.
- A3, Sybil attacker. Controls the aggregator and tries to register or impersonate identities so as to outnumber honest peers.
- A4, compromised gateway. Holds a valid enrolled secret and uploads arbitrary parameters. A4 is a Byzantine client; up to f such nodes are assumed.

The cloud aggregator is trusted for availability and for executing the aggregation rule, but is not given raw payloads. We exclude a compromised aggregator, optimised model-poisoning crafted against the specific aggregation rule [20], and adaptive input-space evasion of the classifier (Section V-F).

The layers are deliberately ordered so that each covers a gap the next cannot. Authentication (Section V-C) bounds the number of identities, which is what makes f finite. Admission checks reject malformed or grossly scaled updates before aggregation. Robust aggregation (Section V-B) then contains the f Byzantine peers that remain. Without the first layer the third is vacuous: an attacker who can mint identities owns the median regardless of its robustness guarantees.

B. Poisoning Defence

a) Aggregation rules and their precondition.: Coordinate-wise median and β-trimmed mean [9] (see also Krum [8]) are robust only while honest clients outnumber Byzantine ones per coordinate:

$$K \geq 2f + 1. \tag{5}$$

On the two-gateway testbed this gives $f = 0$: the median of two values is their mean, so one Byzantine node is half the population. The aggregator therefore computes $f = \lfloor (K - 1)/2 \rfloor$ from the number of distinct enrolled peers that have reported and returns it with the global model, so that a gateway is never told it is protected when it is not.

The aggregator also keys updates by client identity and keeps only the latest upload per peer, so the median is taken across *peers*. An earlier implementation kept a sliding window of the last ten uploads regardless of origin; that computes a median over time, and a persistently malicious node simply occupies a constant share of the window.

An ℓ_2-norm gate sits in front of aggregation. An update whose norm differs from the median norm of the other peers by more than a factor of 4 (configurable) is rejected with HTTP 409 before it enters the median. With fewer than two other peers the reference falls back to the bootstrap model's norm. The gate is a cheap first filter against scaling; it does not detect sign-flips of ordinary magnitude, which is the median's job.

b) Empirical result.: The simulated experiment (seven nodes, two of them uploading sign-flipped, $12\times$ amplified weights from round 4) is reported in Section IV-D, Fig. 4 and Table IV. In short, plain averaging is inverted and ends at 0.263 ROC-AUC, whereas the coordinate-wise median (0.984) and trimmed mean (0.986) are unaffected. The bound (5) holds with margin ($7 \geq 5$).

c) Scope of the claim.: This is a plain sign-flip adversary acting simultaneously and at a fixed boost. The experiment is simulated at $K = 7$ with a single seed; the physical testbed has two gateways and therefore no Byzantine tolerance (Section V-F). We make no claim against attacks optimised for the aggregation rule [20].

C. Authenticated Control Plane

Each gateway is enrolled with a 256-bit secret from the operating system CSPRNG and signs every update with HMAC-SHA256 [34] over a canonical serialisation (sorted keys, fixed separators). HMAC was chosen over a public-key signature to preserve the zero-dependency, $\sim$10 MB edge footprint: `hmac` and `hashlib` are in the Python standard library. The cost is symmetric keys, so the aggregator holds every secret; see Section V-F.

An update passes the following checks in order, and the first failure rejects it:

1) **Signature, timestamp, nonce.** The identity must be enrolled; the signature is compared in constant time before any field is trusted; the timestamp must lie within 120 s of the server clock; the nonce must not have been seen in the last 300 s. Unknown and malformed identities receive the same response, so the endpoint does not reveal which identifiers are enrolled.
2) **Per-identity rate limit.**
3) **Dimension and finiteness.** Wrong length or non-finite values (NaN, infinity) are rejected with HTTP 400.
4) **Norm gate.** Scaled updates are rejected with HTTP 409.
5) **Aggregation** by median or trimmed mean over the latest update per peer.

Note that the replay nonce is consumed during verification, before the rate limiter runs, so a message that draws a rate-limit rejection can never be retried verbatim.

D. Authentication Attack Tests

`test_auth_attacks.py` runs ten adversarial cases against a live aggregator. The first, unsigned, case reproduces the wire format the original aggregator accepted; each case is constructed so that a correct control plane gives a specific status code. The suite replays the current global weights for

TABLE VI
ADVERSARIAL TEST CASES AND THE STATUS CODE A CORRECT AGGREGATOR RETURNS.

#	Attack	Expect	Layer
1	Unsigned update (old format)	401	signature
2	Sybil identity, self-signed	401	enrolment
3	Forged signature, enrolled identity	401	signature
4	Tampered weights, stale signature	401	signature
5	Replay of an accepted update	401	nonce
6	Valid signature, timestamp 1 h old	401	timestamp
7	Second update inside the rate window	429	rate limit
8	Enrolled peer, $50\times$ scaled weights	409	norm gate
9	Enrolled peer, wrong dimension	400	validation
10	Enrolled peer, NaN weights	400	validation

the one case that is meant to be accepted, so running it does not move the deployed model. Table VI lists the cases and expected verdicts.

Cases 1–6 cover A2 and A3: forged, tampered, replayed, stale and Sybil messages. Cases 8–10 cover A4 in its crudest forms, where an enrolled peer sends scaled, malformed or non-finite parameters. Case 5 sends a valid update, confirms it is accepted, and replays it. Because the rate limiter runs before the validation checks, cases expecting 400 or 409 first retry past a 429 with a freshly built message; the harness always rebuilds, never resends, so that a retry cannot trip the replay guard and mask the case under test.

These tests check that each control returns its designed verdict; they are not a proof of security. We do not report a pass count in this paper, because the suite executes against live infrastructure and its per-run output is not archived with the repository. Reproducing it requires the aggregator URL (`FEDZTA_CLOUD`) and the enrolment file.

E. Audit-Pass Fixes

A review of the code against the claims of the draft paper, followed by a hardening pass, produced the changes below.

a) Claim–code mismatches.: The audit found that the original poisoning figure was drawn from hard-coded lists and not from a measurement (replaced by the experiment above); that the Byzantine argument was void at $K = 2$ and the aggregator's window was keyed by time rather than peer (now keyed by client identity, with f reported); and that three mutually inconsistent models were in circulation (a single model file now provides one provenance for the simulation, aggregator, gateway and profiler).

b) Hardening.: The hardening pass addressed the following.

- **Content-Length handling.** Reading the header with `int(headers['Content-Length'])` crashed the handler on a missing or non-numeric value. The header is now read with a default and a `ValueError` yields HTTP 400, in the gateway, the aggregator and the post-quantum aggregator.
- **Slow-connection exhaustion.** Servers use `ThreadingHTTPServer` with a 5 s per-connection timeout, so a Slowloris-style client holding sockets open cannot block legitimate traffic or exhaust the single-threaded loop.
- **Race conditions.** Gateway request/block counters and the aggregator's rejection counters are now updated under locks, since threaded serving made unsynchronised increments lossy.
- **Atomic model publication.** Global model snapshots are written to a temporary file, flushed and `fsync`ed, then moved into place with `os.replace`, so a gateway syncing mid-write never reads a truncated model.
- **Block threshold.** Training-time data augmentation (JSON, form data, varied lengths, prose) shifted the model's intercept; the gateway's block threshold was lowered from 0.5 to 0.15 to compensate. This is an operating-point change, not a security fix, and the trade-off between false positives and recall at that threshold is not quantified (Section IV-G).

F. Limitations and Discussion

a) Byzantine tolerance at deployed scale.: The physical cluster has two gateways, so by (5) its tolerance is $f = 0$. The poisoning results come from a seven-node simulation, not hardware. The norm gate and authentication still apply at $K = 2$, but neither can stop a compromised gateway that uploads plausible-norm, adversarially chosen weights.

b) Adaptive attacks.: Sign-flip poisoning is the simplest attack. Optimised attacks aware of the aggregation rule can defeat median-based defences [20], and we did not evaluate them. Likewise the classifier is a linear model over hashed n-gram and syntactic features and is evadable in principle by padding, entropy shaping or steering trigrams toward lightly weighted buckets; no adaptive input-space evaluation was done. The resilience shown concerns model poisoning, not evasion.

c) Authentication.: Symmetric HMAC keys mean a breach of the aggregator exposes every peer secret, and the scheme provides authenticity and freshness but not non-repudiation. Replay and rate-limit state is held in aggregator memory. Enrolment is a manual provisioning step, which is what bounds f but is also an operational burden. The timestamp window assumes loosely synchronised clocks.

d) Confidentiality.: HMAC authenticates updates; it does not encrypt them. ML-KEM-512 [12] is benchmarked as a component and a post-quantum aggregator variant exists in the repository, but the live weight channel is still JSON over plain HTTP on a trusted network segment, so A2's "store now, decrypt later" capability is not yet neutralised. Differential privacy is specified but not implemented, so we report no ϵ and no measurement of gradient inversion.

e) Statistical scope.: The poisoning experiment is one seeded run with a single attack strength and timing. The attack tests assert designed status codes and were not run as a statistical study.

VI. Conclusion and Future Work

A. Conclusion

We presented a federated [7] Zero-Trust gateway architecture for Industrial IoT and evaluated it under conditions strict enough to be informative: disjoint unique-payload splits and comparison at a matched false-positive budget. Under these conditions the common claim that federation dramatically improves zero-day detection does not hold in general. When nodes differ in their benign traffic profile while sharing a threat landscape, federation matches the strongest isolated silo within run-to-run variance and gives the most balanced cross-domain profile, with privacy preserved at no measurable accuracy cost. Coordinate-wise median aggregation tolerates sign-flipping Byzantine nodes where plain averaging fails, and the resulting gateway, with an authenticated weight channel and a benchmarked ML-KEM key exchange that is not yet integrated into the live path, runs on commodity industrial edge hardware.

B. Deployment Status

Hardening followed a tiered roadmap derived from an adversarial review of the prototype. *Threat visibility*: the edge inferencer now handles `GET`, `POST` and `PUT`, inspects critical headers (`User-Agent`, `Referer`, `Cookie`) and request bodies up to an 8 KB cap, and scores each context (URL, header, body) independently to avoid statistical dilution. *Control-plane authentication*: the cloud aggregator requires HMAC-SHA256-signed updates with timestamps and single-use nonces, checks an enrolment allowlist, rate-limits per identity (nonce consumed before the rate check), and rejects dimensional and norm outliers before median aggregation, closing the Sybil exposure of an unauthenticated update endpoint. *Operations*: classifier weights persist atomically across restarts, global models are versioned for instant rollback, a shadow (monitor-only) mode measures real-world false-positive rate before blocking is enforced, and an explicit fail-open/fail-closed policy governs gateway failures. The pseudo-label feedback loop was replaced by replay of a frozen anchor set to prevent drift.

C. Future Work

Remaining items are: (i) *Data expansion*: SQL-injection recall is limited by the small number of unique payloads, so integrating further public corpora (CTF collections, fuzzing lists, Payloadbox) is the binding next step. (ii) *Non-linear models*: replacing logistic regression with a lightweight gradient-boosted tree ensemble or a two-layer MLP over the same feature vector, while keeping millisecond latency and fixed-size parameters. (iii) *Formal privacy*: differentially private aggregation, quantifying (ϵ, δ) against detection loss. (iv) *Adaptive adversaries*: optimised model poisoning against the median rule and input-space evasion against the n-gram map. (v) *Physical scale-out*: reproducing the Byzantine results on hardware beyond $K = 2f + 1$ gateways. (vi) *Post-quantum integration*: wiring the ML-KEM-derived key into authenticated encryption of the live `/update` and `/weights` path. (vii) *Isolation and adaptivity*: WebAssembly sandboxing of the inferencer against model extraction, and human-in-the-loop active learning to restore online adaptability without reintroducing drift.

Data and Code Availability

The dataset manifests, federated simulation, Byzantine poisoning experiment, pure-Python edge inferencer and testbed instructions are available at https://github.com/eldemaster/FedZTA-Edge.

References

[1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, NIST Special Publication 800-207, 2020.

[2] Z. Zheng *et al.*, "Zero-trust architecture for the internet of things," *IEEE Internet of Things Journal*, vol. 6, no. 5, 2019.

[3] OWASP, "Modsecurity core rule set (CRS)," https://coreruleset.org/, 2023.

[4] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. USENIX LISA*, 1999.

[5] Open Information Security Foundation, "Suricata IDS/IPS," https://suricata.io/.

[6] Gaia-X European Association for Data and Cloud AISBL, "Gaia-x architecture document," Gaia-X, Tech. Rep., 2021.

[7] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NIPS)*, 2017.

[9] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018.

[10] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. ICLR*, 2020.

[11] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[12] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, Federal Information Processing Standard FIPS 203, 2024.

[13] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," *;login:*, vol. 39, no. 6, pp. 6–11, 2014.

[14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010.

[15] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[16] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020.

[17] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proc. ACM CCS*, 2016.

[18] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017.

[19] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A survey on security and privacy of federated learning," *Future Generation Computer Systems*, vol. 115, pp. 619–640, 2021.

[20] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to byzantine-robust federated learning," in *Proc. USENIX Security*, 2020.

[21] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Advances in Neural Information Processing Systems (NIPS)*, 2017.

[22] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DIoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE ICDCS*, 2019.

[23] D. C. Nguyen *et al.*, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.
[24] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symposium on Security and Privacy*, 1996.
[25] K. Wang and S. J. Stolfo, "Anomalous payload-based network intrusion detection," in *Proc. RAID*, 2004.
[26] C. Kruegel and G. Vigna, "Anomaly detection of web-based attacks," in *Proc. ACM CCS*, 2003.
[27] W. B. Cavnar and J. M. Trenkle, "N-gram-based text categorization," in *Proc. Symposium on Document Analysis and Information Retrieval (SDAIR)*, 1994.
[28] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proc. ICML*, 2009.
[29] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
[30] D. J. Bernstein and T. Lange, "Post-quantum cryptography," *Nature*, vol. 549, pp. 188–194, 2017.
[31] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018.
[32] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," NIST, Federal Information Processing Standard FIPS 204, 2024.
[33] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.
[34] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, Tech. Rep. RFC 2104, 1997.
[35] D. Miessler and J. Haddix, "SecLists: the security tester's companion," https://github.com/danielmiessler/SecLists, accessed 2026.